

应用层服务语义

把字节组织成名字、资源与操作

408 · NET-08

1 0. 本册定位

本 Topic 回答：端到端传输只提供 byte stream 或 datagram，应用怎样定义对象、名字、操作、响应和状态，使不同程序能够共同完成 DNS、Web、文件与邮件服务？

本册拥有 application architecture、DNS、HTTP semantics、FTP 与电子邮件的服务分工，以及 application protocol 对 transport service 的选择逻辑。

DHCP 虽以 UDP application protocol 形式运行，但在本项目中由 IP 配置与转发拥有，因为其主要输出是主机进入 IP 世界所需的启动配置。

2 1. 根本问题：transport 不知道“这段 byte 意味着什么”

TCP 可以保证 byte 顺序，却不知道这段 byte 是域名查询、HTTP request、邮件命令还是文件内容。Application protocol 必须定义：

Application object → Name/identifier → Operation → Message representation → Response/state transition

它不仅规定报文格式，还规定动作的语义与交互顺序。相同 transport 可以承载完全不同服务；相同服务语义也可以演进不同 transport bindings。

3 2. 应用架构：谁长期持有状态

3.1 2.1 Client/Server

server 通常在稳定地址/名字上等待请求并持有共享服务状态；client 主动发起交互。集中服务便于管理、一致性和发现，代价是容量、故障和运营中心化。

3.2 2.2 Peer-to-Peer

peer 同时消费和提供资源，容量可随参与者增加，但发现、可用性、信任、NAT 穿越和一致性更复杂。

C/S 与 P2P 是状态与职责的组织方式，不等同于 TCP/UDP，也不由“是否有服务器进程”一个表象决定。

4 3. DNS：把一个集中表拆成层次化授权系统

4.1 3.1 为什么单一 hosts 文件失败

全网共享一份名字表会遇到：更新冲突、分发延迟、单点容量与管理边界。DNS 的生成链是：

Global flat file fails → Hierarchical namespace → Delegated zones → Distributed authoritative servers → Resolver

Domain namespace 是名字树；zone 是某个 authority 实际管理的数据边界。树的结构与管理分区相关，但 domain 不等于一台主机或一个物理网络。

4.2 3.2 四类对象

- stub resolver: 应用/OS 发起查询的本地接口；
- recursive resolver: 代用户完成后续查询并缓存；
- authoritative name server: 对某 zone 的数据负责；
- root/TLD/other delegated servers: 通过 referral 把查询逐步引向 authority。

4.3 3.3 一次 cache miss 的解析轨迹

```
application asks local resolver
-> recursive resolver checks cache
-> root referral identifies TLD servers
-> TLD referral identifies authoritative servers
-> authoritative answer returns records
-> resolver caches answer/referrals under TTL
-> result returns to application
```

迭代查询返回“我不知道最终答案，但下一步问谁”；递归查询把继续查找的责任交给被询问方。实际 client-to-recursive 常请求递归，resolver 对 authority 链执行迭代式追踪。

4.4 3.4 Cache 的正确性边界

cache 用旧信息换低延迟和低查询负载。TTL 限制可复用时间，但 DNS 不是瞬时全局一致系统：更新在 TTL 生命周期内可能与缓存旧值并存。

DNS 并非只能用 UDP。经典普通查询常用 UDP；响应截断、zone transfer 等情形可使用 TCP，现代部署还存在加密传输。408 做题要服从题设，不能把“DNS=UDP”写成协议语义。

5 4. HTTP: 操作资源的统一语义

5.1 4.1 WWW: 分布式超文档系统

WWW 不是某一个 protocol 或“所有 Internet 服务”的同义词。它至少包含：browser/client、Web server、由 URL/URI 标识的 resource、HTML 等 document/representation，以及把文档与其他资源连接起来的 hyperlink。HTTP 定义 client 与 server 操作这些资源的请求/响应语义；DNS 可把 URL 中的 host name 解析为 IP；transport 再承载 HTTP message。

URL identifies resource → DNS locates host → HTTP requests representation → HTML links reveal dependencies

“网页”常是一个主文档与图片、样式、脚本等多个对象组成的依赖图，不是一条 HTTP response 必然完整返回的单文件。URL 提供标识与定位语法，HTML 提供超文本表示，HTTP 提供操作/传输语义；

三者不能互相替代。

HTTP 的核心对象不是“网页文件”，而是由 URI 标识的 resource。消息传输的是 resource 的 representation, method 表达 client 希望对目标执行的语义。

Target resource → Method semantics → Request metadata/content → Status → Representation/metadata

5.2 4.2 Resource 不等于 representation

同一 resource 可根据时间、语言、编码或内容协商产生不同 representation。response body 是某次选定表示，不等于资源本身的全部身份。

5.3 4.3 Method 属性改变重试决策

- safe: 语义上只读，例如 GET/HEAD;
- idempotent: 重复执行与执行一次具有相同预期 effect，例如 PUT、DELETE 以及 safe methods;
- POST 通常不假定 idempotent。

这些属性不是礼貌标签，而是 cache、proxy、client 在失败后能否自动重试的重要决策依据。

5.4 4.4 Stateless 不等于“服务没有状态”

HTTP stateless 表示每个 request 的语义不依赖 server 必须记住此前 protocol request context。应用仍可通过 cookie、token、database 和 server-side session 建立业务状态。

5.5 4.5 语义与连接版本分离

- HTTP/1.1 和 HTTP/2 通常在 TCP 上承载;
- HTTP/3 使用 QUIC over UDP;
- 核心 method、status、representation 和 cache semantics 在版本间保持共同定义。

因此“HTTP 使用 TCP”是特定版本/部署的 transport mapping，不是 HTTP resource semantics 本身。

6 5. Web 性能从依赖图生成

取得一个 URL 可能依次触发：DNS、transport/security establishment、HTTP request/response、HTML parsing、更多 subresource requests。

总时间不能只数“几个 RTT”，必须先画依赖关系：哪些步骤串行、哪些 connection 可复用、哪些 object 可并行、哪些命中 cache。

Persistent connection 减少重复建立连接；multiplexing 允许多个 request/response 共享连接并发推进；cache 用 freshness/validation 状态减少传输。三者解决的瓶颈不同。

7 6. FTP：控制状态与数据传输分离

FTP 的母问题是跨异构系统可靠操作和传输文件。它使用持久 control connection 交换命令与会话状态，并为目录/文件数据建立独立 data connection。

这种 out-of-band control 让命令不被大文件数据阻塞，也带来额外连接管理、主动/被动模式和 NAT/firewall 适配成本。FTP 的两条 TCP connections 是应用层设计，不是 TCP 自动生成的“控制通道”。

8 7. 电子邮件：提交、转发、存储、读取不是一个协议

邮件系统至少有：user agent、mail server、message transfer 和 mailbox access。

```
sender user agent
-> submission / SMTP transfer
-> one or more mail servers
-> recipient mailbox
-> POP3 or IMAP access
-> recipient user agent
```

- SMTP 负责提交/服务器间 push 式传送；
- POP3 提供较简单的 mailbox download/access 模型；
- IMAP 更强调服务器端 mailbox 状态与多设备同步；
- MIME 扩展 message representation，使非 ASCII 内容和附件可被邮件体系承载。

“发邮件”和“收邮件”必须分开建模，不能把所有过程归给 SMTP。

9 8. 408 应用协议流程：消息、状态与停止条件

9.1 DNS：递归责任与迭代 referral 分开

stub 向 recursive resolver 发送 query (name、type、class、transaction ID 等)；resolver 先查 cache。未命中时，它依次向 root、TLD 和 authoritative server 发迭代查询，referral 通过 NS 及必要的 glue/address 信息指出下一位 authority；最终 answer 或 negative result 按 TTL 缓存并返回 stub。收到 response 时必须核对 query identity/匹配字段；TTL 到期后旧 cache 不再直接回答。UDP 是经典普通查询的常见承载，截断或 zone transfer 等分支可转 TCP，不能把承载写成唯一语义。

9.2 HTTP：请求语义与连接时延分开

一次交互至少推进：

```
method + target + fields + optional content → server processing → status + fields + optional representation
```

性能题先画依赖：DNS 是否命中、transport 是否已有连接、连接能否复用、对象是否串行/并行、内容大小和链路速率。Persistent connection 减少重复建连，cache 减少请求/传输，multiplexing 改变并发关系；三者不能用同一个“少一个 RTT”口诀替代。

9.3 FTP：控制连接决定数据连接怎样建立

client 先建立持久 control connection 并完成认证/命令。需要目录或文件数据时，主动模式由 server 从约定 data port 连接 client 提供的 endpoint；被动模式由 server 提供监听 endpoint，client 主动连接。一次 data transfer 完成后数据连接可关闭，control state 继续存在。端口常数和命令细节以题设为准，稳定边界是 control/data 两条状态轨迹。

9.4 Mail: 提交、relay 与 access 三段

sender user agent 把 message 提交给 SMTP server；服务器通过 DNS MX 等信息选择下一跳并用 SMTP relay 推送到 recipient mail server；message 进入 mailbox 后，recipient 再用 POP3/IMAP 等访问协议读取。SMTP reply 驱动提交/转发状态，MIME 只定义可承载的内容表示；它们都不拥有 mailbox access。停止条件应分别写“已被下一 SMTP hop 接受”和“用户已从 mailbox 访问”，不能合并成一次端到端 ACK。

Owner 边界

DHCP 虽常列在应用层协议表中，但本项目由 NET04 Own 其配置生命周期；NET-I01 只调用 DNS、TCP 和 HTTP 的输出，不重复本节报文机制。

10 9. 怎样选择 transport service

应用选择的不是协议名偏好，而是约束组合：

应用需求	需要观察的 transport 能力
不能丢 byte、允许额外延迟	reliable ordered stream
保留 message boundary、自定义恢复	datagram
极低启动延迟	connection setup cost
多路流并发且减少队头阻塞	multiplexing semantics
广播 bootstrap	local datagram/broadcast support
安全身份与机密性	security layer/secure transport

“实时应用一定用 UDP”“可靠应用一定用 TCP”都过度简化。应用可在 UDP 之上实现可靠/拥塞控制，或为穿越部署环境选择 TCP；判断必须回到服务需求与实现责任。

11 9. 概念边界

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Domain name	≠ IP address	人/组织命名与网络定位；DNS 维护映射	把 DNS 当透
Domain	≠ Zone	名字树节点/子树与 authority 管理边界	授权与层次结
Recursive query	≠ Iterative query	被询问方完成解析；被询问方返回 referral	DNS 报文轨
Resource	≠ Representation	被标识对象与某次可传输表示	缓存和内容
HTTP stateless	≠ Application has no state	协议请求独立与业务状态存在可并存	认为登录/cc
HTTP semantics	≠ HTTP transport version	method/status/resource 与具体连接承载分离	把 HTTP/3
SMTP	≠ POP3/IMAP	邮件传送与 mailbox access	邮件全流程
FTP control connection	≠ FTP data connection	命令状态与内容传输分离	端口/连接数

12 10. 做题调用协议

1. 写应用对象：name、resource、file、message 还是 lease/config；
2. 写谁持有 authority/state，谁只是 cache/client；
3. 画 request/response 或 push/pull 的事件方向；
4. DNS 题区分 recursion、iteration、authority、cache TTL；
5. HTTP 题区分 resource、representation、method 和 connection；
6. 邮件题拆成 submission/transfer/storage/access；
7. 性能题画依赖 DAG，再数不可并行的 RTT/transfer；
8. 最后检查 transport 选择是否来自应用约束而非口诀。

13 11. 贯穿母例：输入 URL 后第一个 HTTP request 为什么还不能立刻发

```
URL identifies scheme + authority + target
-> resolver needs server IP (unless cached)
-> IP forwarding needs next hop and local MAC (unless cached)
-> transport/security context may need establishment
-> HTTP request names resource and method
-> response carries status + selected representation
-> representation may reveal more resource dependencies
```

URL、domain、IP、port、MAC 分别服务不同作用域。把它们压成一句“浏览器访问服务器”，会隐藏几乎所有可诊断的状态交接。

14 12. 高频 First Divergence

- 把 DNS 说成从 root 一次返回最终 IP：漏掉 delegation/referral；
- 认为 cache 命中永远是最新真相：忽略 TTL 与弱一致时间窗；
- 写“DNS 用 UDP，HTTP 用 TCP”作为完整机制：把常见承载当唯一语义；
- 把 HTTP stateless 理解成网站不能登录：混淆 protocol context 与 application state；
- 把 response body 等同 resource：混淆对象与 representation；
- 用 SMTP 从 mailbox 下载邮件：没有分开 transfer 与 access。

15 13. 一页压缩与复原问题

Name an application object → Choose operation semantics → Represent in messages → Map onto transport → E

1. DNS 怎样用 delegation 替代全球单表？
2. cache 为什么同时提高性能并引入旧信息窗口？
3. HTTP resource 与 representation 为什么必须分开？
4. stateless protocol 怎样支撑有状态登录业务？
5. SMTP、POP3/IMAP 和 MIME 分别拥有邮件过程的哪一段？

16 14. 来源与校正说明

- 归档笔记《应用层-FTP,SMTP,POP3,DHCP》提供传统应用协议覆盖；《应用层-DNS》《应用层-WWW与 HTTP》为空文件，未被当作已有正文；
- DNS 的 hierarchy、delegation、resolver、authority 与 cache 边界依据 RFC 1034 校正；
- HTTP 的 stateless semantics、resource/representation 和跨版本语义边界依据 RFC 9110 校正；
- DHCP 已按当前 Ownership 移入 IP Topic，不在应用册重复拥有。